Name: Mim Akter Mou
IT: 22057

1. Demostrate how a child class can access a protected member of its parent class within the same package.

Ans:
In Java, a protected member of a class is accessible
- Within the same package by any class.
- In subclasses, even if they are in a different package.

Example:
Two classes in the same package:

1. Parent.Java

```
package mypackage;
public class Parent{
    protected String message = "Hello";
    }
```

2. Child.Java

```
Package mypackage;
public class Child extends Parent{
    public void displayMessage(){
        System.out.println(message);
        }
```

```
    public static void main(String[] args){
        Child c= new Child();
        c.displayMessage();
    }
}
```

Explain with example what happens when the child class is in a different package.

Ans:
Two class in different package:

1. Parent.Java

```
package Cow;
public class Parent{
    protected String message = "Hello";
}
```

2. Child.Java

```
Package Childpkg;
import cow.Parent;
public class Child extends Parent{
    public void showMessage(){
```

```
        System.out.println(message);
    }
    public static void main(String[] args){
        Child child = new Child();
        child.showMessage();
        Parent p = new Parent();
    }
}
```

A sub class in a different package can access protected member of the parent class only through inheritance, not through the object of the parent.

2. Compare abstract class and interfaces in terms of multiple inheritance. When would you prefer to use an abstract class and when an interface?

Points	Abstract Class	Interface
Definition	Cannot be instantiated; contains both abstract (without implementation) and concrete methods (with implementation)	Specifies a set of methods a class must implement; methods are abstract by default.
Implementation Method	Can have both implemented and abstract methods.	Methods are abstract by default; Java 8 can have default and static methods.
Inheritance	Class can inherit from only one abstract class.	A class can implement multiple interfaces.
Access Modifiers	Methods and properties can have any access modifier (public, protected, private	Methods and properties are implicitly public
Variables	Can have member variables (final, non-final, static, non-static	Variables are implicitly public, static and final (constants)

Use Abstract class :

- You I want to share common base functionality on state (fields) among related classes.
- You I expect future subclasses to inherit behavior.
- You I want to enforce a contract with partial implementation.

Example :

```
abstract class Animal{
        void eat()
            System.out.println("Eating...");
        }
        abstract void makesound();
}
```

Use Interface :

```
interface Flyable{
        void fly();
}
```

- I want to define a contract across unrelated classes.
- I need to support multiple inheritance of types.
- I am building APIs for things like Serializable, Comparable, Comparable, Runable.

3. How does encapsulation ensure data security and integrity? Show with a BankAccount class using private variables and validated methods such as setAccount-Number(String), setInitialBalance(double) that rejects null, negative or empty values.

Ans: Encapsulation ensures data security and integrity by:

- Hiding internal data using private variables so that they can't be accessed or modified directly from outside the class.
- Exposing controlled access through public methods that validate input, thus preventing invalid or inconsistend data.

Example: BankAccount class

```
public class BankAccount {
    private String accountNumber;
    private double balance;
    public void setAccountNumber(String account
     Number) {
        if (accountNumber == null || accountNumber
          trim().isEmpty()) {
            throw new IllegalArgumentExpetio
            ("Account number cannot to null or
             empty.");
        }
        this.accountNumber = accountNumber;
}
```

```
    public void setInitialBanance (double balance){
        if (balance <0) {
            throw new IllegalArgumentException
            ("Initial balance cannot be negative.");
        }
        this.balance = balance;
    }
    public String getAccountNumber(){
        return accountNumber;
    }
    public doble getBalance(){
        return balance;
    }
}
```

Example: Usage

```
public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();
```

```
        account.setAccountNumber("ABC123");
        account.setInitialBalance(5000.0);
        System.out.println("Account:" + account.getAccountNumber
        ());
        System.out.println("Balance: $" + account.getBalance());
    }
}
```

4.1) Find the kth smallest element in an ArrayList.

Ans:

```
import java.util.*;
public class kthSmallestElement {
    public static int findkthSmallest(List<Integer> list, int k)
        if(list == null || list.size() < k || k <= 0) {
            throw new IllegalArgumentException("Invalid
            input: list is too small or k is out of bounds");
        }
        Collections.sort(list);
        return list.get(k-1);
    }
```

```
public static void main(String[] args) {
    ArrayList<Integer> numbers = new ArrayList
    <>(Arrays.asList(12, 3, 5, 7, 19, 1));
    int k = 3;
    int kthSmallest = findkthSmallest(numbers, k);
    System.out.println(k+"rd smallest element is "+
    kthSmallest);
  }
}
```

ii) Create a TreeMap to store the mappings of words to their frequencies in a given text.

```
import java.util.*;
public class WordFrequencyTreeMap{
    public static void main(String[] args){
        String text = "Java is powerful. Java is
        fast. Java is everywhere!";
```

```
String[] words = text.toLowerCase().split("\\W+");
TreeMap<String, Integer> wordFreq = new TreeMap<>();

for (String word: words){
    if (!word.isEmpty()){
        wordFreq.put(word, wordFreq.getOrDefault
        (word, 0) + 1);
    }
}

System.out.println("Word Frequencies (Sorted):");
for (Map.Entry<String, Integer> entry: wordFreq.
entrySet()){
        System.out.println(entry.getKey() +
        "=" + entry.getValue());

    }
}

}
```

iv) Create a TreeMap to store the mapping of student IDs to their details

Ans:

```
import java.util.*;
class Student{
    private String name;
    private String department;
    private double gpa;
    public Student(String name, String department, double gp
        this.name = name;
        this.department = department;
        this.gpa = gpa;
}
@Override
public String toString(){
    return "Name:"+name+", Dept:"+
    department+", GPA:"+gpa;
}
}
```

```
public class StudentMapExample{
    public static void main(String[] args){
        TreeMap<Integer, Student> studentMap = new
        TreeMap<> ();
        studentMap.put(1002, new Student("Bob", "CSE", 3.6));
        System.out.println("Student Records(Sorted by ID):");
        for(Map.Entry<Integer, Student> entry: studentMap.
        entrySet()){
            System.out.println("ID:"+entry.getKey()
            "->"+entry.getValue());
        }
    }
}
```

5. Developing a multithreading-based project to simulate a car parking management system with classes namely - RegistranParking - Represents a parking request made by a car ; ParkingPool. Acts as a shared synchronized queue; Parking Agent Repnents a thread that continuosly checks the pool and parks cars from the queue and a Mainclass - Simulates N cars arriving concurrently to request parking.

```
import java.util.concurrent.*;
import java.util.*;
class RegistranParking {
    Private final String carNumber;
    public RegistranParking (String carNumber){
        this.carNumber = carNumber;
    }
```

```
public String getCarNumber(){
    return carNumber;
}
}
class ParkingPool {
    private final BlockingQueue<RegistranParking>
    queue = new LinkedBlockingQueue<>();
    public void requestParking(RegistranParking car)
        System.out.println("Car" + car.getCarNumber
        "requested parking.");
        queue.offer(car);
    }
    public RegisterParking takeCar() throws Interru
    tedException {
        return queue.take();
    }
}
```

```
class ParkingAgent extends Thread{
    private final ParkingPool pool;
    private final int agentId;
    public ParkingAgent(ParkingPool pool, int agentId){
        this.pool = pool;
        this.agentId = agentId;
    }
public void run(){
    try{
        while (true){
            RegistranParking car = pool.takeCar();
            Thread.sleep(1000);
            System.out.println("Agent "+agentId+
            "parked car"+car.getCarNumber()+".
        }
    }catch (InterruptedException e){
        System.out.print("Agent "+agentId+
        "stopped.");
    }
}
}
```

```
public class MainClass{
    public static void main(String[] args) throws
    InterruptedException{
        ParkingPool parkingPool = new ParkingPool();
        ParkingAgent agent1 = new ParkingAgent(parkingPool,
        1);
        ParkingAgent agent2=new ParkingAgent(parkingPool,2);
        agent1.start();
        agent2.start();
        List<String> carNumbers = Arrays.asList("ABC123",
        "XYZ456","LMN789");
        ExecutorService executor = Executors.newFixed
        ThreadPool(carNumbers.size());
        for(String number: carNumbers){
            executor.execute(()->{
                RegistrarParking car = new
                RegistrarParking(number);
                ParkingPool.requestParking(car);
            });
    }
```

```
        executor.shutdown();
        executor.awaitTermination(5, TimeUnit.SECONDS);
        agent1.interrupt();
        agent2.interrupt();
    }
}
```

6. DOM vs SAX Parser

Feature	DOM Parser	SAX Parser
Parsing Style	Tree-based (loads entire XML into memory)	Event-based (reads XML ~~seqe~~ sequentially)
Memory Usage	High-entire XML is stored in memory	Low-does not store XML in memory.
Speed	Slower for large files	Faster for large files
Navigation	Easy-can traverse back and forth	Difficult-forward only processing
Modification	Supported can modify XML nodes	Not supported read only.
Ease of Use	Easier to implement and understand	More complex due to event handling.

<config>
    <setting name = "language"> English</setting>
</config>

Dom is ideal because it allows:

- Reading the entire XML structure.
- Modifying values
- Saving back to the file.

SAX is preferred because:

- dont need the entire file in memory
- Process each <transactions> as it comes
- avoid memory overload and get faster performa

Perefer SAX over DOM:

I am processing a large log file in XML format and only need to extract certain tags.

2. Virtual DOM

- A lightweight in memory tree representing the real DOM.
- React uses it to determine the minimal updates needed.

Feature	Traditional DOM	Virtual DOM
Updates	Immediate, direct	Batched, optimized
Performance	Slower on frequent changes	Faster with diffing
Control	Manual via JS	Declarative via React
DOM Manipulation	High cost	Minimal and efficient

Example:

```
function Greet(props) {
    return <h1> Hello, {props.name}! </h1>;
}
```

- React apps with frequent UI updates.
- Optimized re rendering of lists, forms, dynamic content.

8. Event Delegation:

Event Delegation is a techeique where a ~~signa~~ single event listener is attached to a parent element, which handles events triggered by its child elements, even if they are added dynamically.

Use-

- Better perfonmance → Fewer listeners-lower memory usage
- Dynamic Handling → Works with elements added later
- Cleaner code → One listener ✓ instead of many

Example: HTML:

```
<div id="button-container"></div>
<button onclick="addButton()">Add Button</button>
```

JavaScript:

```
const container = document.getElementById("button-con
    container");
container.addEventListener("click", function (event){
    if(event.target.tagName === "BUTTON"){
        alert("You clicked:" + event.target.textConte
    }
```

```
function addButton(){
    const btn = document.createElement("button");
    btn.textContent = "Button" + (container.children.
    length + 1);
    container.appendChild(btn);
}
```

9. Regular Expressions: Java provides the java.util.regex package to perform pattern matching using regular expressions (regex). This is widely used for input validation like emails, phone numbers, passwords etc.

Use Regex:

1. Define the regex pattern
2. Use Pattern.compile(regex) to compile it.
3. Use matcher(input) to apply the pattern.
4. Use matcher.matches() to validate.

Code:

```
import java.util.regex.*;
public class EmailValidator {
    public static void main(String[] args){
        String email = "user@exampl.com";
        String regex = "[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$";
        pattern pattern = Pattern.compile(regex);
        Matcher matcher = pattern.matcher(email);
        if( matcher.matches()){
            System.out.println("Valid email address.");
        }else{
            System.out.println("Invalid email address.");
        }
    }
}
```

10. Custom Annotations: Custom annotations are user-defined metadata used to mark code elements. With @Retention(RUNTIME) they can influence runtime behavior via reflection.

Use -

- Access control
- Configuration
- Loggin or testing
- Custom runtime logic

1. Define Annotation:

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface RunIfAdmin{
    String role() default "user";
}
```

2. Use It:

```
public class AccessControl{
    @RunIfAdmin(role = "admin")
    public void deleteUser(){
```

```
        System.out.println("User deleted.");
    }
}
```

3. Process with Reflection

```
for(Method method : obj.getClass().getDeclaredMethods()){
    if (method.isAnnotationPresent(RunIfAdmin.class)){
        RunIfAdmin ann = method.getAnnotation(RunIfAdmin.class);
        if(ann.role().equals("admin")){
            method.invoke(obj);
        }
    }
}
```

11. Singleton Pattern:

Ensures that a class has only one instance and provides a global point of access to it.

Solves:

- Prevents multiple instances
- Ensures consistent shared state across the app

⊞ Basic (Non-thread-safe) Implementation

```
public class Singleton{
    private static Singleton instance;
    private Singleton(){}
    public static Singleton getInstance(){
        if (instance == null) instance = new
            Singleton();
            return instance;
    }
}
```

⊞ Thread-Safe Implementations.

```
public static synchronized Singleton getInstance
    if (instance == null) instance = new Singleton()
    return instance;
}
```

⊞ Double-checked Locking

```
private static volatile Singleton instance;
public static Singleton getInstance(){
    if (instance == null){
        synchronized (Singleton.class){
```

```
        if (instance == null) instance = new Singleton();
    }
  }
  return instance;
}
```

12. JDBC: Java Database Connectivity is an API that enables Java applications to connet to and communicate with relational databases using SQL.

```
package db;
import...
public class DBConnection{
    public static Connection getConneation(){
        try{
            Class.forName("org.postgresql.Driver");
            return DriverManager.getConnection(
              "Jdbc:postgresql://localhost:5432/
               abcd", "postgres", "root");
            }catch(Exception e){
                e.printStackTrace();
                return null;
            }
        }
}
```

13. Servlets and JSPs work:

i) User submits a request

ii) Servlet receives it, processes data and interacts with the Model.

iii) Model returns the data.

iv) Servlet forwards the result to JSP to render HTML.

i) Use case: Model

```
public class Student{
    private int id;
    private ~~int~~ String name;
    public Student(int id, String name){
        this.id = id;
        this.name = name;
    }
    public int getId(){
        return id;
    }
    public String getName(){
        return name;
    }
}
```

Controller:

```
protected void doGet(HttpServletRequest req, HttpServle
Response res)
        throws ServletException, IOException {
    List<Student> students = List.of(
        new Student(1, "Muskan");
        new Student(2, "Minad"));
    req.setAttribute("StudentList", students);
    req.setRequestDispatcher("Student.JSP").forward(req,
}
```

View:

```
<%@ page import = "Java.util.*, your.package.Student" %>
<ul>
<%
  List<Student> list = (List<Student>) request.getAttribute
  ("studentlist");
  for (Student s: list){
%>
  <li> ID: <%= s.getId() %>, Name: <%= s.getName()
<%} %>
</ul>
```

14. Life cycle of a Java Servlet:

i) Load class → Servlet class is loaded by container

ii) Instantiate → Servlet object is created

iii) init() → called once for initialization

iv) service() → called on every request

v) destory() → called once before servlet is unloaded.

Servlet Method

i) init():

- called once when the servlet is initialized.
- Used for resource allocation, reading config parameters etc.
- Not called again unless the servlet is reloaded.

ii) service():

- Called for every request
- Handles routing to doGet(), doPost() etc.
- Must be efficient to serve multiple concurrent requests.

iii) destroy():

- called once before the servlet is unloaded.
- Used for resource cleanup, like closing DB connections or file handles.

Servlets handle concurrent request:

- By default, a single instance of the servlet handles all requests.
- Multiple threads may call service() simultaneously.
- Servlet container uses multithreading, not multiple instances.

Thread Safety Issues:

In Java servlets, the servlet container create one instance of the servlet class and handle multiple client requests using separate threa If the servlet uses shared resources it can lead to thread safety issues when those resources are accessed or modified concur

15. In Java Servlet, the container creates one instance of each servlet and uses threads to handle multiple requests.

Thread Safety Issue:

```
public class CounterServlet extends HttpServlet{
  private int count = 0;
  protected void doGet(HttpServletRequest req,
    HttpServletResponse res)
        throws IOException{
    count++;
    res.getWriter().println("Visitor num:"+count);
  }
}
```

Use Synchronized to Make it Thread-Safe

```
public class SynchronizedCounterServlet extends
  HttpServlet{
  private int count = 0;
  protected void doGet(HttpServletRequest req,
    HttpServletResponse res)
```

```
        throws IOException{
    synchronized (this){
        count++;
        res.getWriter().println("Visiton num : " +count);
    }
  }
}
```

16. MVC (Model-view-controller) is a software design pattern that separates a web application into three main components: model, view, controller

MVC separates concerns:

- model: knows nothing about the UI, only handles data.
- View: knows nothing about logic, only renders output.
- Controller: Acts as a bridge. fetches data, passes it to the view.

Student Registration System

- Model:

```
public class student{
    private String name, email;
}
```

- DAO:

```
public void registen (Student student){

}
```

- Controllen :

```
protected void doPost(...){
    Student student = new Student();
    student.setName(req.getParameter("name"));
    student.setEmail(req.getParameter("email"));
    new StudentDAO().registen(student);
    req.setAttribute("student", student);
    req.getRequestDispatcher("success.jsp").
        forward(req, res);
}
```

- View :

```
<h2>Registration Successful</h2>
<P>Name: <%= student.getName() %></P>
<P>Email: <%= student.getEmail() %></P>
```

17. Servlet controller role in Java EE

- Receives client requests
- Interacts with the Model to process or fetch data.
- Stores data in request scope.
- Forwards the request and data to a JSP for rendering.
- The JSP generates the HTML response using that data

Servlet:

```
Protected void doGet(HttpServletRequest req,
 Http.ServletResponse res)
        throws ServletException, IOException {
    List<String> student = List.of("Alice", "Bob");
    req.SetAttribute("studentlist", students);
    req.getRequestDispatcher("students.JSP").forwar
    (req.res);
}
```

JSP:

```
<%@ page import = "Java.util.List" %>
<ul>
<%
```

```
List<String> students = (List<String>)
request.getAttribute("studentList");
for(String s : students){
%>
<li><%= s %></li>
<% } %>
</ul>
```

18. Session tracking allows a server to remember a ~~att~~ user's state across multiple HTTP request.

Cookies:

- Stores small name value pairs in the client browser.
- `cookie c = new Cookie("username", "mim");`
  `response.addCookie(c);`

Pros:

- Persistent
- Simple to use

Cons:

- Size limit (~4kB)
- Can be disabled by user
- Sent with every request

2. Rewriting

Appends session data

Example: response.sendRedirect("dashboard.jsp?user=mim");

Pros: • works without cookies

Cons: • Exposes data in URL

• Manual effort to append session info to every link

• Session lost if URL shared/bookmarked.

19.1. Session Creation

• Created when user logs in:

HttpSession session = request.getSession();

2. Session ID

• Server assigns a unique ID JSESSIONID stored in the browser as a cookie.

3. Session Access Across Requests

• On each request, the browser sends JSESSI allowing the server to retrieve session data

String user = (String) session.getAttribute("user

4. Session Data Storage
• Stores user-specific data
session.setAttribute("username", "mim123");

Session Timeout Handling
1. Automatic Timeout
• Session expires after inactivity
• Configured via:
<session-config>
<session-timeout>30</session-timeout>
</session-config>

2. Manual Invalidation
• On logout or forced exit:
session.invalidate();

21. 1) Intercepts Request
• Acts as the front controller
• Receives all incoming HTTP requests.
2) Finds the Handler
• Uses HandlerMapping to determine the

connect @Controller method based on:

- URL
- HTTP method

3. Calls the Controller

- Uses a HandenAdapter to invoke the controller method.

4. Returns Model and View Name

- controller returns:
  return "dashboard"
- And optionally adds attributes to the Model

5. Resolves View

- Uses ViewResolver to map logical view name to actual file:
  - eg "dashboard" -> /WEB-INF/views/dashboard.

6. Renders View

- The view is rendered with model data.

7. Sends Response

- DispatcherServlet returns the rendered page to the browser.

22. i) Security - Prevents SQL Injection
• Uses parameter placeholders (?) and automatically escapes inputs.
• Prevents malicious input from breaking SQL logic.

ii) Performance - Precompiled SQL
• SQL query is parsed and compiled once, reused with different values.
• Faster when executing the same query multiple times.

iii) Maintainability - Cleaner code.
• Easier to set parameters without string concatenation
• Avoids syntax errors and improves readability.

Insert Record Using PreparedStatement

Table: create table users (
id int primary key auto_increement;
name varchar(50), email varchar(100));

Java code:

```
import java.sql.*;
public class InsertUser{
    public static void main(String[] args){
        String url = "jdbc:mysql://localhost:3306/postgre_sql";
        String user = "root";
        String password = "root";
        String sql = "INSERT INTO users (name, email)
        values ("Mim", "mimakternou25@email.com)";
        try(Connection c = DriverManager.getConnection
            (url, user, password);
            ps.setString(1, "John Doe");
            ps.setString(2, "john@email.com");
            int rows = ps.executeUpdate();
            System.out.println(rows + "rows(s) inserted.
        }catch(SQLException e){
            e.printStackTrace();
        }
```

29. Maven in Spring Boot

Dependency Management

- Dependencies are declared in pom.xml
- Maven downloads libraries from Maven central.
- Transitive dependencies are resolved automatically.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Typical pom.xml Structure

```
<project>
  <modelVersion>4.0.0</modelVersion>

  <parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.2.0</version>
  </parent>
<groupId>com.example</groupId>
<artifactId>demo</artifactId>
<version>1.0.0</version>
<packaging>jar</packaging>
```

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

<build>
   <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>Spring-boot-maven-plugin</artifact
    </plugin>
   </plugins>
  </build>
</project>
```

27. Model class:

```
public class User {
    private String name;
    private int age;

}
```

2. Controller : UserController.java

```
@RestController
@RequestMapping("/api/user")
public class UserController{
    private List<User> userList = new ArrayList<>();
    @public List<User> getAllUsers(){
        return userList;
    }
    @postMapping
    public String addUser(@RequestBoby User user){
        userList.add(user);
        return "User add:" + user.getName();
    }
}
```

3. Main Class : DemoApplication.Java

```
@SpringBootApplication
public class DemoApplication{
    public static void main(String[] args){
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

30. Maven vs Gradle

Feature	Maven	Gradle
Syntax	XML (pom.xml)	Groovy/kotlin DSL
Verbosity	More verbose	More concise and readable
Performance	Slower builds	Faster
Custom Tasks	Limited	Easy with Groovy
Learning Curve	Easier for beginners	Steeper due to DSL flexibility
IDE Support	Excellent (IntelliJ) (Eclipse)	Excellent (especially with IntelliJ)
Multi-module Support	Strong	Strong

Example:

```
@RestController
public class HelloController {
    @GetMapping("/hello")
    public String hello() {
        return "Hello, Spring Boot!";
    }
}
```